Université Paris Cité, Campus Saint-Germain-des-Près
UFR de Mathématiques et Informatique
45, rue des Saints-Pères 75006 Paris

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Algorithmique et Programmation 2

Correction du partiel du 16 mars 2026 — durée: 1h30

Tout document interdit. Les calculatrices et les téléphones portables, même prévus à titre d'horloge, sont également interdits.

MERCI DE BIEN INDIQUER VOTRE GROUPE DE TD SUR VOTRE COPIE

Tous les exercices sont indépendants. On précisera bien la signature (seulement le type des entrées/sorties) de chaque fonction Python proposée. Ces signatures seront notées globalement sur 1 point.

Barème sur 22,5 points (en comptant le point global pour les signatures)

Exercice 1. **(au total sur 6 points)** Pour chaque ligne d'instructions ci-dessous, donner le contenu de la variable `x` **(type et valeur)** après exécution de la ligne considérée, ou expliquer le problème si l'instruction produit une erreur. **Expliquer votre réponse (une réponse correcte sans explication ne sera pas comptabilisée).**

```
1  x = bin(3) + 0b1011
2  x = round(3.6)*int(-1.7)
3  x = float(10**1000)
4  j = 1; x = (j + 2j)**2
5  L = [3,1,5,2]; x = sorted(L, reverse=True).index(1)
6  x = list(range(10)).append(0)
7  x = [1]; y = [x]; y[0] = 3
8  x = set(range(1,3)) < set(range(2,4))
9  x = ({1,2} | {2,3}) & {3,4}
10 x = set("123"); x.discard(1)
11 x = 1 in frozenset([1,2,3])
12 x = {1:2, 3:4, 5:6}; x = x[x.get(4,1)]
```

(1) Erreur car on ne peut appliquer l'opérateur + entre une chaîne de caractère et un entier (`bin(3)` retourne `"0b11"` et `0b1011` retourne 11).

(2) Le résultat est -4, de type `int`, car `round(3.6)` retourne 4 (arrondi à l'entier le plus proche) et `int(-1.7)` retourne -1 (arrondi en enlevant la partie dédcimale).

(3) Erreur car le nombre entier 10^{1000} est trop grand pour être converti en `float`.

(4) Le résultat est `-3+4j`, de type `complex`, en effet $x = (1+2i)^2 = 1 - 4 + 4i = -3 + 4i$.

(5) L'instruction `sorted(L, reverse=True)` produit la liste `[5,3,2,1]` (tri décroissant), donc le résultat est `x=3` (int), l'indice de la première occurence de 1 dans cette liste.

(6) L'instruction `list(range(10))` produit la liste `[0,1,2,3,4,5,6,7,8,9]`, à laquelle la méthode `append` est appliquée. Cette méthode modifie la liste mais ne renvoie rien, le résultat est donc `x = None` (de type `Nonetype`).

(7) `x = [1]` (type list), c'est le cas au départ et cela ne change pas car l'instructiony$[0] = 3$ modifie la valeur de `y[0]` (qui devient un entier), pas le contenu de la liste `x` anciennement référencée par `y[0]`.

(8) `x = False` (booléen), car il est faux que l'ensemble $\{1, 2\}$ est strictement inclus dans l'ensemble $\{2, 3\}$.

(9) `x = {3}` (type set), en effet on calcule $(\{1, 2\} \cup \{2, 3\}) \cap \{3, 4\} = \{1, 2, 3\} \cap \{3, 4\} = \{3\}$.

(10) `x = {"1","2","3"}` (type set), car l'instruction `set("123")` fabrique un ensemble avec les composants élémentaires (caractères) de la chaîne de caractères "123", et la méthode `discard()` utilisée ensuite ne modifie pas `x` puisque l'entier 1 n'est pas un élément de `x`.

(11) `x = True` (booléen), car 1 est bien élément de l'ensemble (de type frozenset) $\{1, 2, 3\}$.

(12) `x = 2` (int), en effet comme 4 n'est pas une clé du dictionnaire `x` l'instruction `x.get(4,1)` retourne la valeur 1, et donc finalement x prend la valeur associée à la clé 1, c'est-à-dire 2.

Exercice 2. **(au total sur 2 points)** On considère une fonction `f` définie avec l'en-tête suivant:

```
def f(x=1, y=True, z="0"):
```

Pour chaque ligne ci-dessous, déterminer si l'appel à la fonction `f` est valide (dans ce cas, préciser la valeur de chaque argument dans `f`) ou si l'appel est invalide (dans ce cas, expliquer pourquoi).

```
1 f("0", True, 1)
2 f()
3 f(z=2, True)
4 f(1, False, y="1")
```

(1) appel correct, `x="0", y=True, z=1`
(2) appel correct, `x=1, y=True, z="0"` (valeurs par défaut)
(3) erreur, une spécification positionnelle (2e argument, True) arrive après une spécification par mot-clé (`z=2`).
(4) erreur, l'argument `y` est spécifié deux fois, une fois de manière positionnelle (2e argument, `False`), et une fois par mot-clé (`y="1"`).

Exercice 3. **(au total sur 2 points)**

(1) **(sur 0,5 point)** Quelle est la complexité, dans le cas le pire, de l'algorithe du tri rapide ?

La complexité pour trier une liste à n éléments est $O(n^2)$ (voir cours).

(2) **(sur 1,5 point)** Soit `f` une fonction Python qui implémente le tri rapide en prendant comme pivot le premier élément. Dessiner l'arbre des appels récursifs associé à l'appel `f([4,2,8,6,3,1,5])`. Combien y a-t-il, au total, d'appels à la fonction `f` ?

Il y a en gros deux arbres possibles, selon que l'on considère que la condition d'arrêt de `f` (pas d'appel récursif) s'applique pour les listes de taille 0 ou 1, ou uniquement pour les listes vides. Dans le premier cas, on obtient l'arbre:

```
                    f([4, 2, 8, 6, 3, 1, 5])
                      /                  \
                     /                    \
            f([2, 3, 1])             f([8,6,5])
              /      \                 /    \
             /        \               /      \
        f([1])    f([3])        f([6,5])  f([])
                                  /   \
                                 /     \
                             f([5])   f([])
```

et il y a donc exactement **9** appels à la fonction `f`.

Dans le 2e cas, toute liste non vide produit deux appels récursifs, ce qui conduit à un total de **15** appels à la fonction `f`:

```
                    f([4, 2, 8, 6, 3, 1, 5])
                      /                  \
                     /                    \
            f([2, 3, 1])             f([8,6,5])
             /       \                 /    \
            /         \               /      \
        f([1])      f([3])        f([6,5])  f([])
        /   \        /   \          /  \
       /     \      /     \        /    \
    f([])  f([]) f([])  f([])   f([5])  f([])
                                 /   \
                                /     \
                             f([])   f([])
```

Exercice 4. **(au total sur 3,5 points)**

On considère une suite $(u_n)_{n\in\mathbb{N}}$ définie par $u_0 = 3$ et la relation de récurrence

$$\forall n \in \mathbb{N}, \quad u_{n+1} = \begin{cases} n + 2u_n & \text{si } u_n \text{ est pair,} \\ 3 - u_n & \text{sinon.} \end{cases}$$

(1) **(sur 0,5 point)** Calculer u_4.

On a successivement:

$$\begin{aligned} u_1 &= 3 - u_0 = 0 & (n = 0, u_0 \text{ impair}) \\ u_2 &= 1 + 2u_1 = 1 & (n = 1, u_1 \text{ pair}) \\ u_3 &= 3 - u_2 = 2 & (n = 2, u_2 \text{ impair} \\ u_4 &= 3 + 2u_3 = 7 & (n = 3, u_3 \text{ pair}) \end{aligned}$$

(2) **(sur 1,5 point)** Écrire une fonction **récursive** `suite1(n)` qui prend en entrée un entier naturel n et renvoie la valeur de u_n.

```
def suite1(n):
    """ int -> int """
    if n==0:
        return 3
    u = suite1(n-1)
    if u%2==0:
        return n-1 + 2*u
    return 3 - u
```

Attention, l'implémentation ci-dessous n'est pas correcte, car elle effectue 2 appels récursifs :

```
def suite1(n):
    """ int -> int """
    if n==0:
        return 3
    if suite1(n-1)%2==0:
        return n-1 + 2*suite1(n-1)
    return 3 - suite1(n-1)
```

(3) **(sur 1,5 point)** Écrire une fonction **non récursive** `suite2(n)` qui prend en entrée un entier naturel n et renvoie la liste $[u_0, u_1, \ldots, u_n]$.

```
def suite2(n):
    """ int -> list """
    u = [3]
    for k in range(1, n+1):
        if u[k-1]%2==0:
            u.append(k-1 + 2*u[k-1])
        else:
            u.append(3 - u[k-1])
    return u
```

Exercice 5. **(au total sur 6 points)**

(1) **(sur 1,5 point)** Écrire une fonction `compte(L)`, qui prend en entrée une liste `L` formée d'entiers, et retourne un dictionnaire dont les clés sont les entiers de la liste `L`, et la valeur d'un entier `k` est l'indice dans `L` de sa dernière occurence dans `L`. Par exemple, `compte([1,3,2,2,1])` retourne le dictionnaire `D={1:4, 2:3, 3:1}`.

```
def compte(L):
    """ list -> dict """
    D = dict()
    for i in range(len(L)):
        D[L[i]] = i
    return D
```

(2) **(sur 2 points)** Écrire une fonction `code(s1, s2)`, qui prend en entrée deux chaînes de caractères `s1` et `s2`, et retourne un dictionnaire `D` dont les clés sont les caractères de `s1`, et la valeur associée à chaque caractère de `s1` est le caractère correspondant à la même position dans la chaîne `s2`. La fonction retourne `None` lorsque les deux chaînes ne sont pas de même taille, ou lorsque toutes les occurences d'un même caractère de `s1` n'ont pas la même correspondance dans `s2`.
Par exemple, `code('abca', 'xxox')` retourne le dictionnaire `D={'a':'x', 'b':'x', 'c':'o'}`, et `code('abca', 'uuxy')` retourne `None`.

```
def code(s1, s2):
    """ str x str -> dict ou None"""
    D = dict()
    n = len(s1)
    if len(s2)!=n:
        return None
    for i in range(n):
        if s1[i] in D and D[s1[i]]!=s2[i]:
            return None
        D[s1[i]] = s2[i]
    return D
```

(3) **(sur 1,5 point)** Écrire une fonction génératrice `pos(L)`, qui prend en entrée une liste, et délivre, dans l'ordre, tous les termes positifs (au sens large) de la liste. Par exemple, si `L = [1, 0, -2, 4, -1, 6, -3]`, alors `pos(L)` doit délivrer successivement les valeurs 1, 0, 4, et 6.

```
def pos(L):
    """ list -> generator """
    for x in L:
        if x>=0:
            yield x
```

(4) **(sur 1 point)** Écrire une fonction `compose(f, g)`, qui prend en entrée deux fonctions à un argument `f` et `g` et retourne la fonction composée $f \circ g$. Par exemple, si `f = round` et `g = lambda x:x**2`, alors `compose(f, g)` doit retourner la fonction `lambda x:round(x**2)`.

```
def compose(f, g):
    """ function x function -> function """
    return lambda x: f(g(x))
```

Exercice 6. **(sur 2 points)**

On appelle partition d'un entier $n > 0$ toute liste formée d'une suite finie décroissante d'entiers strictement positifs dont la somme vaut n. Par exemple, pour $n = 4$ il y a exactement 5 partitions possibles :

`[1, 1, 1, 1], [2, 1, 1], [2, 2], [3, 1], [4]`

On peut remarquer que toute partition de n non réduite à un élément est formée par la concaténation de la liste à un élément `[k]` (pour un certain entier $k > 0$) et d'une partition de $n - k$ (avec une condition de compatibilité à trouver).

Utiliser cette propriété pour écrire une fonction génératrice **récursive** `partition(n)` qui prend en entrée un entier $n > 0$, et délivre toutes les partitions de n.

On peut remarquer que :

- il y a une seule partition de n à un élément, c'est `[n]`;
- toute autre partition s'écrit donc comme la concaténation de la liste `[k]` (pour un certain $k \in \{1, 2, \ldots n-1\}$) et d'une partition L de $n - k$. Une construction de ce type n'est cependant valide que si $L[0] \leqslant k$ pour respecter la condition de décroissance.

On obtient ainsi le code suivant :

```
def partition(n):
    """ int -> generator """
    for k in range(1,n):
        for L in partition(n-k):
            if L[0]<=k:
                yield [k]+L
    yield [n]
```